

Hochschule
Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

Rendern einer Website

Web Engineering 1 (319300)

Vorlesung 02

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-10-13

Agenda

Agenda

- Rendern einer Website
- DOM
- HTML
- CSS
- JavaScript

HTML zu DOM

Rückblick

Vorlesung 1: URL → DNS → HTTP-Request → **Response-Body ist da** Der Server hat HTML geschickt. **Was macht der Browser daraus?**

Parsing-Pipeline

Bytes	→	**Zeichen**	→	**Tokens**	→	**DOM-Baum**
Welche Kodierung?		Unicode / UTF-8		Tags, Attribute, Text		lebendes Objektmodell

Jede Stufe wird durch den WHATWG HTML §13.2 Standard algorithmisch definiert.

Stufe 1: Bytes → Zeichen

Bytes werden anhand ihrer Kodierung zu Zeichen umgewandelt

Woher weiß der Browser, wie er Bytes dekodieren muss?

- **BOM** (Byte Order Mark, erste 2-2 Bytes)
- **Prescan** der ersten 1024 Bytes nach `<meta charset="...">`
- **HTTP-Header** `Content-Type: text/html; charset=utf-8`
- Alternativ: Heuristik (aber unzuverlässig!)

Stufe 1: Bytes → Zeichen

Bytes werden anhand ihrer Kodierung zu Zeichen umgewandelt

Woher weiß der Browser, wie er Bytes dekodieren muss?

- **BOM** (Byte Order Mark, erste 2-2 Bytes)
- **Prescan** der ersten 1024 Bytes nach `<meta charset="...">`
- **HTTP-Header** `Content-Type: text/html; charset=utf-8`
- Alternativ: Heuristik (aber unzuverlässig!)

Faustregel: immer UTF-8 deklarieren

Der Standard verlangt es und 98,9% des Webs nutzen es

Stufe 1: Bytes → Zeichen

Bytes werden anhand ihrer Kodierung zu Zeichen umgewandelt

Woher weiß der Browser, wie er Bytes dekodieren muss?

- **BOM** (Byte Order Mark, erste 2-2 Bytes)
- **Prescan** der ersten 1024 Bytes nach `<meta charset="...">`
- **HTTP-Header** `Content-Type: text/html; charset=utf-8`
- Alternativ: Heuristik (aber unzuverlässig!)

Faustregel: immer UTF-8 deklarieren

Der Standard verlangt es und 98,9% des Webs nutzen es

Fehlende Charset-Angabe

München wird zu MÃ¼nchen

Stufe 2: Tokenisierung

- Tokenizer ist eine Zustandsmaschine mit ~80 möglichen Zuständen
- Token-Typen: `DOCTYPE`, Start-Tag, End-Tag, Attribute, Kommentar, Zeichen

```html
<p class="intro">Hallo</p>
```
HTML

```
Start-Tag "p" [class="intro"] → Zeichen "Hallo" → End-Tag "p"
```

Tags sind Syntax, Tokens sind strukturierte Daten

Stufe 3: Baumkonstrukion

- **Insertion Modes**: in head, in body, in table … + Stack offener Elemente besimmten, wo ein Token landet
- Styntax ist dabei nebensächlich

```HTML
<p>Ein Absatz
<div>Ein Block
```

- `div` in `p` ist nicht erlaubt → Parser schließt `p` automatisch
- **Ergebnis**: Zwei Geschwister entstehen, kein Fehler

Fehlerbehandlung & DOCTYPE

- Parse-Fehler sind nie fatal: Browser produziert immer einen DOM-Baum
- Regeln für Wiederherstellung stehen im WHATWG Standard (ca. 90 Seiten)
- Ergebis ist für jeden Browser gleich

DOCTYPE

`<!DOCTYPE html>` wählt den Rendering-Modus: Standards vs. Quirks Mode

- Quirks Mode: anderes CSS/Layout-Verhalten
- Fehler werden in jedem Modus korrigiert
- Bowser validiert nichts

Streaming & Zusammenfassung

Streaming

- Parsing beginnt mit den ersten Bytes
- Browser wartet nicht auf das ganze Dokument

Zusammenfassung

- DOM ist die lebende Wahrheit - der Quelltext ist nur ein Vorschlag
- Zwei Rollen für den DOM
  - Eingabe für das Rendering
  - API für JavaScript

Was ist der DOM?

Was ist der DOM?

- DOM → **D**ocument **O**bject **M**odel
- Zwei Aufgaben
  - **Rendering Input**: der DOM (+CSSOM) das Layout
  - **Programmierungs-API**: Interface, dass JavaScript nutzt um mit der Seite
- Ein Live Modell: Zeigt den aktuellen Status, nicht die Quelle
- DOM ≠ Markup

Node Model

- **Alles** ist eine Node
- Haupttypen: `Document`, `DocumentType`, `Element`, `Text`, `Comment`, `DocumentFragment`
- Elemente $\neq$ Tags:
  - Tag ist Syntax
  - Element ist ein Runtime Objekt mit:
    - Identität
    - Namespace
    - Attributen
    - Position im Baum

API Familien

- **Auswahl**: `document.getElementById`, `querySelector` / `querySelectorAll`
- **Traversal**: `parentNode`, `children`, `firstElementChild`
- **Mutation**: `createElement`, `append`, `remove`, `insertAdjacentHTML`
- **Properties vs. Attribute**:
  - `href` (Property)
  - `getAttribute("href")` (Attribute)
- **Content Zugriff**:
  - `textContent`
  - `innerHTML`

Events (Vorschau) & Performance

Events

- DOM ist auch ein Eventsystem
- `addEventListener`
- bubbling/capturing: Events wandern durch den Baum
- `event.target`: wo passiert es
- `event.currentTarget`: wer hört zu

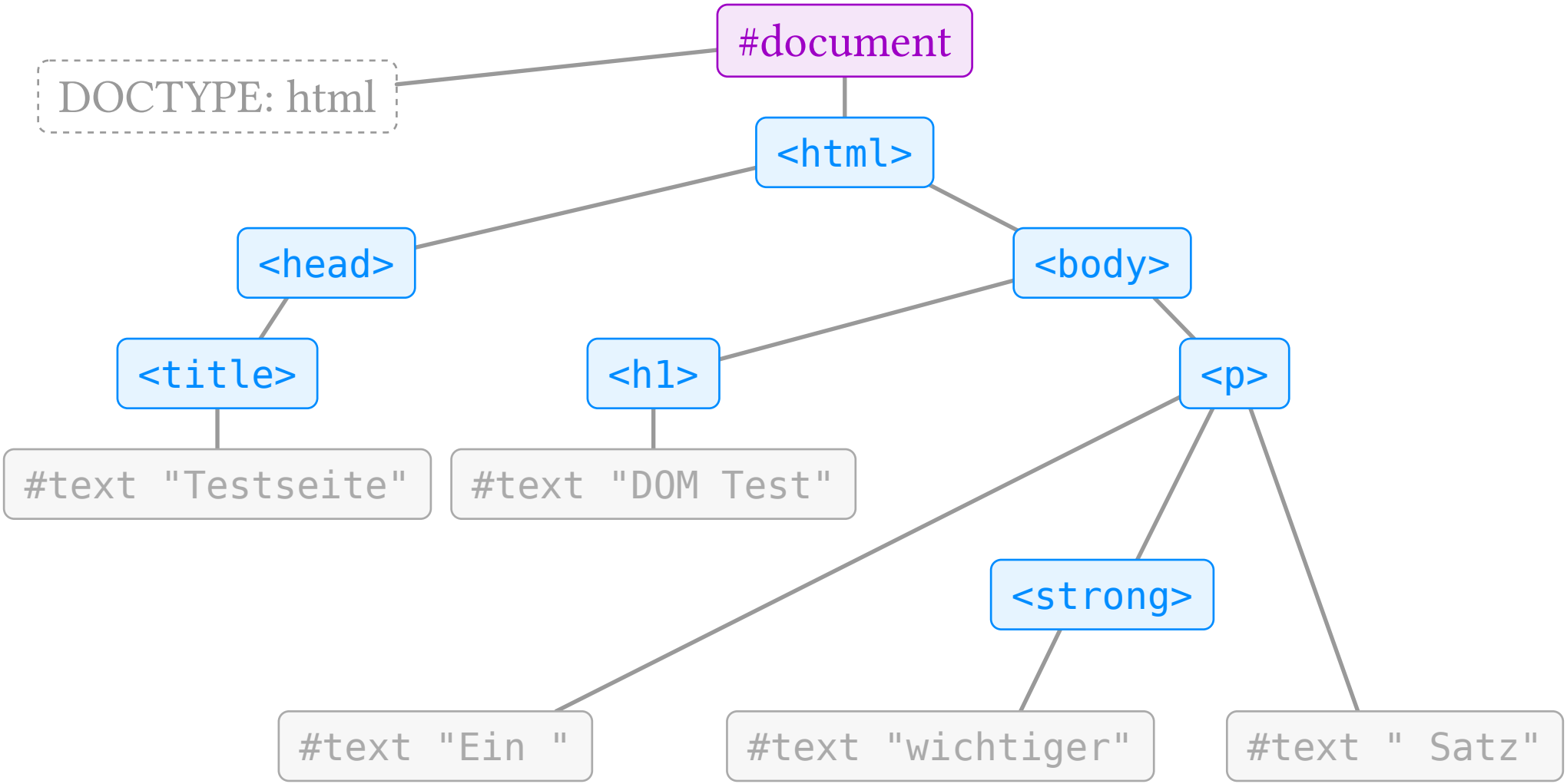
Performance

- DOM ist teuer: viele Nodes und tiefe Bäume verlangsamen Parsing
- Scripting des DOMs führt zu Renderarbeit
- Frameworks existieren, um direkte DOM Manipulation zu vermeiden

Beispiel

```html
<!DOCTYPE html>
<html>
  <head>
    <title>Testseite</title>
  </head>
  <body>
    <h1>DOM Test</h1>
    <p>Ein <strong>wichtiger</strong> Satz.</p>
  </body>
</html>
```

Beispiel


#document
DOCTYPE: html
<html>
<head>
<body>
<title>
<h1>
<p>
#text "Testseite"
#text "DOM Test"

#text "Ein "
#text "wichtiger"
#text " Satz"

HTML

Semantisches HTML

- HTML beschreibt Bedeutung, nicht das Aussehen
- Browser, Screenreader, Suchmaschinen lesen Bedeutung, nicht Optik

> HTML sagt, was etwas ist und CSS sagt später, wie es aussieht

HTML

```html
<!-- bedeutungslos -->
<div class="header-top">...</div>

<!-- bedeutungsvoll -->
<header>...</header>
```

Strukturelemente

```html
<body>
    <header> Kopfzeile, Logo, Navigation </header>
    <nav> Hauptnavigation </nav>
    <main> Inhalt </main>
    <article> Eigenständiger Beitrag </article>
    <section> thematischer Abschnitt </section>
    <aside> Randnotiz </aside>
    <footer> Fußzeile, Impressung </footer>
</body>
```

- **Landmarks** für Screenreader-Navigation & Suchmaschinen
- `main` nur einmal pro Seite
- `article` als in sich geschlossener Inhalt

Text

Überschriften-Hierarchie

- `h1-h6` bilden die **Gliederung** des Dokuments
- Große Sprünge zwischen Kategorien sollten vermieden werden
- Überschriften nicht nach Schriftgröße wählen
- Optik kann mit CSS geregelt werden

Textsemantik

- `p`, `ul`/`ol`/`li` für Absätze und Listen
- `strong`/`em` für Text mit Bedeutung wichtig oder betont
- `b`/`i` nur für Darstellung
- `img` mit `alt`-Text
- `a` mit Linktext

Barrierefreiheit

- Screenreader navigieren über Landmarks & Überschriften
- Gute Semantik ermöglicht Zugänglichkeit
- `alt`-Texte beschreiben Bildinhalte
- Bedienbarkeit über Tastatur ermöglichen
  - Markieren des fokussierten Elements über Outline
  - Springen zur nächsten Textbox/Element mit Tab

CSS

CSS einbinden

HTML

```html
<!-- 1. Inline: Antipattern -->
<p style="color: red">...</p>

<!-- 2. <style> im head: ok für kleine Seiten -->
<style> p { color: red; } </style>

<!-- 3. Extern: der empfohlene Standardweg -->
<link rel="stylesheet" href="style.css">
```

- **Inline** bekämpft die Kaskade & wiederholt sich auf jeder Seite
- **Extern**: eine Datei kann auf vielen Seiten genutzt werden und Caching
- `<link>` früh im `<head>` platzieren
- CSS blockiert Rendering, nicht parsing

Selektoren

```
1    Selektor { Eingenschaft: Wert; }
```
CSS

```
1    p { color: #333; } /* Element */
2    .card { padding: 1rem; } /* Klasse */
3    #navigation { top: 0; } /* ID */
```
CSS

- Klassen sind der Normalfall
- IDs sind oft zu spezifisch

```
1    nav a { ... } /* Nachfahre: Alle Links in nav */
2    ul > li { ... } /* direktes Kind */
```
CSS

- Pseudoklassen für Zustand: `:hover`, `:focus`, `:nth-child()`
- Pseudoelemente als Teil des Inhalts: `::before`, `::after`

Vererbung

- Vererben von: `color`, `font-family`, `font-size`, `line-height`
- Keine Vererbung von: Box- & Layout-Eigenschaften (`width`, `padding`, …)

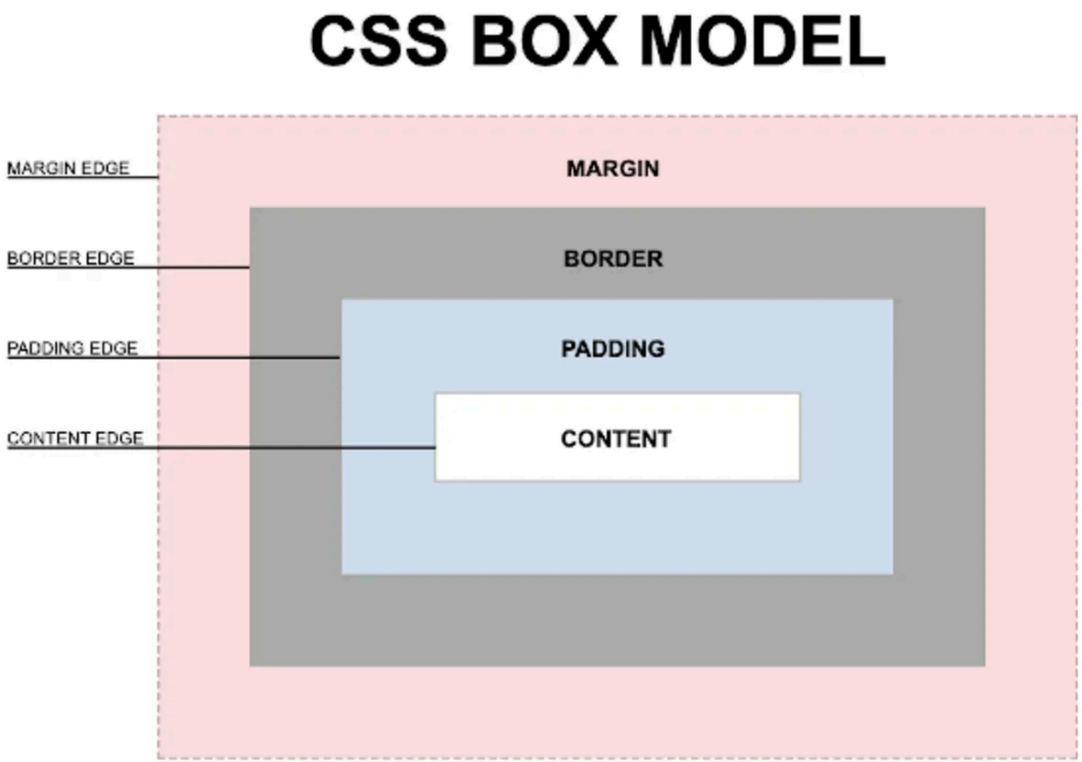
```css
body {
  font-family: "Inter", sans-serif; /* erbt an alles */
  color: #222;
}

.card {
  padding: 1rem; /* wird nicht vererbt */
}
```

- Steuerung: `inherit` (erzwingen), `initial` (originaler Wert), `unset`
- Praxistipp: Styling, dass alle Elemente nutzen werden einmal für `body` definieren

Box Model

Aufbau



- `padding`: Innenabstand
- `margin`: Außenabstand
- Farbige Darstellung in den DevTools

Flexbox

- Lösen eines typischen Problems des Box-Modells: Elemente flexibel in Reihe oder Spalte anordnen, verteilen oder ausrichten
- Zwei Rollen:
  - **Flex-Container**: Das Parentelement `display: flex`
  - **Flex-Items**: Die Children, die automatisch zu Flex-Items werden, wenn der Parent ein Flex-Container ist

Flexboxen Achsen

- Es existieren Haut- und Querachse

```css
.container {
  display: flex;
  flex-direction: row; /* Hauptachse horizontal, Querachse ist vertikal */
}
```

- `justify-content` betrifft immer die Hauptachse
- `align-items` betrifft immer Querachse

Flexboxen Beispiele

Ausrichtung entlang der Hauptachse

```css
.container {
  display: flex;
  justify-content: flex-start; /* Standard, alles am Anfang */
  justify-content: center; /* mittig */
  justify-content: space-between; /* erstes/letztes Element am Rand, Rest gleichmäßig verteilt */
  justify-content: space-around; /* gleicher Abstand um jedes Element */
  justify-content: space-evenly; /* exakt gleicher Abstand überall, inkl. Rändern */
}
```

Flexboxen Beispiele

Ausrichtung entlang der Querachse

```css
.container {
  display: flex;
  align-items: stretch; /* Standard, Items füllen die Container-Höhe */
  align-items: center; /* vertikal zentriert bei row */
  align-items: flex-start; /* oben */
  align-items: flex-end; /* unten */
}
```

Flexboxen Beispiele

Umbruchverhalten

CSS

```css
.container {
  display: flex;
  flex-wrap: nowrap; /* Standard: alles in eine Reihe quetschen */
  flex-wrap: wrap; /* Umbruch in neue Zeile, wenn kein Platz mehr ist */
}
```

Flexboxen Beispiele

Abstände zwischen Items

```css
.container {
  display: flex;
  gap: 1rem;
}
```

Flexboxen Beispiele

Verhalten einzelner Items

```css
.item {
  flex-grow: 0;   /* Standard: wächst nicht, auch wenn Platz frei ist */
  flex-shrink: 1; /* Standard: darf schrumpfen, wenn zu wenig Platz ist */
  flex-basis: auto; /* Startgröße vor Verteilung des freien Platzes */
}

.item {
  flex: 1; /* entspricht flex-grow: 1; flex-shrink: 1; flex-basis: 0; */
}
```

Flexbox vs. Grid

- **Flexbox** ist eindimensional
  - Eine Reihe oder eine Spalte
  - Umbruch ist Nebeneffekt
  - Kein echtes 2D-Raster
  - **Use Case**: Navbars, Button-Gruppen, Card-Reihen
- **Grid** ist zweidimensional
  - Zeilen und Spalten werden gleichzeitig geplant
  - **Use Case**: Komplettes Seitenlayout

JavaScript

JavaScript & DOM

```html
<script src="app.js" defer></script>
```

HTML

- Callback zum Parsing: klassisches `<script>`-Tag blockiert den Parser
- `defer` wird nach dem Parser ausgeführt (Reihenfolge wie im Dokument geschrieben)
- `async` wird parallel geladen und ausgeführt sobald es fertig ist

Faustregel: `defer` im `<head>`, wenn Reihenfolge wichtig ist

JavaScript & DOM

```js
// Auswahl durch CSS-Selektoren
const card = document.querySelector(".card");
const allElements = document.querySelectorAll("nav a");

// Erstellen und hinzufügen von neuen Elementen
const newElement = document.createElement("p");
newElement.textContent = "New JavaScript element";
document.body.append(newElement);

// Verändern und Entfernen von Elementen
card.classList.add("highlight");
card.remove();
```

```html
<a id="link" href="https://hszg.de">HSZG</a>
```

```js
link.getAttribute("href"); // Quelltext String
link.href; // Property
link.href = "https://...";
```

- Attribute: Zustand im Quelltext
- Properties: der lebende Zustand des Objekts

Deshalb: `input.value` Property ändert sich beim tippen. `value` Attribut bleibt dem Quelltext treu

```js
const button = document.querySelector("button");

button.addEventListener("click", (event) => {
  console.log(event.target); // wo es passiert ist
  console.log(event.currentTarget); // wer zuhört
});
```

- **Capturing**: Event wandert nach unten → Target → Bubbling: nach **oben**
- **Event-Delegation**: Ein Listener am Elternteil verwaltet viele Kinder

```js
document.querySelector("ul").addEventListener("click",
(e) => {
  if (e.target.matches("li")) e.target.classList
})
```

Zusammenfassung

Schichtmodell

Schicht	Technologie	Verantwortung
Inhalt	HTML	was etwas ist
Darstellung	CSS	wie es aussieht
Verhalten	JavaScript	was es tut

Warum Schichten trennen?

- Wartbarkeit
- Caching
- Einfachere Teamarbeit